Survey Forms (Google Forms Clone)

Build Guide: Project Creation, File Map, Integration and Requirements

Written by Kasinadh Sarma

13 October 2026

Contents

1 Overview **2**

2 Requirements **2**

2.1 Software . . . 2

2.2 Packages . . . 2

2.3 Why these libraries . . . 3

2.4 Accounts and configuration . . . 4

3 How the Layers Connect **5**

4 Step-by-Step Build **5**

5 What Was Verified **11**

6 Known Limits **11**

7 Source Code **12**

7.1 Configuration . . . 12

7.2 Shared API schemas . . . 13

7.3 Server foundations . . . 17

7.4 Repository layer . . . 19

7.5 API routes . . . 26

7.6 Client library . . . 31

7.7 Components . . . 34

7.8 Pages . . . 44

1 Overview

This guide explains, in order, how to create the project, which files are created or changed at each step, and how the layers connect. The application lets a user build a survey, share a public link, collect validated responses, and view the submissions.

Stack. Next.js (App Router), Tailwind CSS v4, react-hook-form, Zod, `@better-fetch/fetch`, Hono (inside a Next.js route handler), Prisma 7 with PostgreSQL as the primary database, and Cloud Firestore (through the Firebase Admin SDK) as a live mirror of that data and as the store for uploaded files.

Project layout. The project uses the root `app/` layout (no `src/` folder). Section 5 says exactly what was checked and what was not.

Core idea. The API Zod schemas, kept one file per concern in `lib/schemas/`, are imported by both the browser and the API. The same rules therefore run in the form, in the Hono validator, and before anything reaches a database.

Decisions in this version.

- **Hono, not Elysia.** Hono runs inside a Next.js route handler, giving one project and one deployment.
- **Postgres is the source of truth; Firestore mirrors it.** Every write goes to Postgres through Prisma first and is then copied to Firestore under the same ids. If Postgres is unreachable, reads fall back to Firestore. Section 3 explains the design and its limits.
- **Files live in Firestore, not in a storage bucket.** A new Firebase bucket may need a paid plan, so uploaded files are stored as documents in a Firestore collection. This caps a file at 700 KB (Firestore documents are limited to 1 MiB). Moving to a real bucket later changes only `lib/server/repo/files.ts` and the two file routes.

2 Requirements

2.1 Software

Requirement	Version	Purpose
Node.js	22 or newer (24.18 used here)	Runtime
npm	10 or newer (11.16 used here)	Package manager
PostgreSQL	Local, Docker, Neon or Supabase	Primary database used by Prisma
Firebase project	Cloud Firestore enabled, Admin SDK service account	Mirror database and file store
Git	Any recent version	Version control

2.2 Packages

These are the versions installed in this project on 13 October 2026, read from `node_modules`.

Package	Version	Role
`next`	16.4.0	Framework (Turbopack, Cache Components on by default)
`react`, `react-dom`	19.3.0	UI
`tailwindcss`, `@tailwindcss/postcss`	4.3.3	Styling (v4, CSS-first)
`postcss`	8.5.30	Used by the Tailwind plugin
`zod`	4.6.7	Shared schemas and types
`react-hook-form`	7.90.0	Form state
`@hookform/resolvers`	5.9.1	Connects Zod to react-hook-form
`@better-fetch/fetch`	1.3.2	Typed HTTP client
`hono`	4.13.13	API framework
`@hono/zod-validator`	0.9.1	Request validation middleware
`@prisma/client`, `prisma`	7.10.0	Database client, migrations and generation
`@prisma/adapter-pg`, `pg`	7.10.0, 8.23.1	Postgres driver adapter (required by Prisma 7)
`firebase-admin`	14.5.0	Firestore access from the server
`dotenv`	18.0.7	Loads `.env` for `prisma.config.ts`
`typescript`, `eslint`, `prettier`	5.9.3, 9.39.5, 3.9.10	Types, lint, formatting

2.3 Why these libraries

Each package is here for one job, and the list is kept short on purpose.

Package	Why it is used
`next`, `react`, `react-dom`	One framework for pages, routing and server rendering, and the host for the API route handler.
`tailwindcss`, `@tailwindcss/postcss`, `postcss`	Utility-first styling without a separate stylesheet system. v4 is CSS-first, and the PostCSS plugin keeps styling independent of the bundler.
`zod`	One schema per concern, shared by the browser and the API, so the form, the validator and the database layer apply the same rules and the TypeScript types are inferred from them.
`react-hook-form`	Fast form state for the survey builder and response forms, with few re-renders.
`@hookform/resolvers`	Lets react-hook-form use the Zod schemas directly instead of duplicating rules.
`@better-fetch/fetch`	A TypeScript-first wrapper over native `fetch`: typed responses and one error shape, without the weight of Axios.
`hono`	A small, fast web framework that runs inside a Next.js route handler, so the API ships in the same project and deployment.
`@hono/zod-validator`	Validates request bodies, params and queries in Hono with the shared Zod schemas.
`@prisma/client`, `prisma`	A type-safe database client generated from the schema, plus migrations.
`@prisma/adapter-pg`, `pg`	Prisma 7 requires a driver adapter; this is the node-postgres one for PostgreSQL.
`firebase-admin`	Server-side access to Firestore, used as the live mirror and file store.
`dotenv`	Loads `.env` for `prisma.config.ts`, which Next.js does not do.
`typescript`, `eslint`, `prettier`	Static types, lint rules and consistent formatting.

Prisma versions must match. The npm `latest` tag of `prisma` pointed to an 8.0 release candidate while `@prisma/client` was 7.10.0. The install commands in Step 2 pin the three Prisma packages to 7.10.0. Change all three together when you upgrade.

2.4 Accounts and configuration

- A PostgreSQL connection string (`DATABASE_URL`).
- A Firebase project with a Cloud Firestore database (the `(default)` database; the free Spark plan is enough).
- A service account key for the Firebase Admin SDK: project id, client email and private key (Firebase console, Project settings, Service accounts, Generate new private key).
- A random token for the sync endpoints (`SYNC_TOKEN`).

3 How the Layers Connect

Layer	What it does
`components/`	Forms, tables and inputs. Validate with the shared Zod schemas before sending.
`lib/fetch.ts`	Typed client; every endpoint has input, output, params and query schemas.
`lib/server/routes/`	Hono routes. Validate again on the server; return one error shape.
`lib/server/repo/forms.ts`, `submissions.ts`	The only entry points the routes use. They combine the two databases.
`lib/server/repo/postgres/`	Prisma code: the source of truth.
`lib/server/repo/firestore/`	Firestore code: the mirror and the read fallback.
`lib/server/repo/files.ts`	Uploaded files, stored in Firestore.
`lib/server/repo/sync.ts`	Compares and repairs the two databases.

Writes. `createForm`, `deleteForm` and `createSubmission` run in Postgres first. Only after Postgres has committed does the same row go to Firestore, under the same id. A failure of the Firestore write is logged and does not fail the request, because the response is already correct; the sync endpoint repairs the difference.

Reads. Reads come from Postgres. If Postgres throws (for example it is unreachable), the same read runs against Firestore. A "not found" from Postgres is a real answer and is not retried.

Firestore layout. A form is one document in `forms/{id}` with its questions embedded and a `responseCount`. Each submission is a document in the subcollection `forms/{id}/submissions/{id}`. Uploaded files are documents in `files/{id}`.

Counter. When a submission is mirrored, a transaction writes the document and increments `responseCount` only if the document did not exist, so mirroring the same submission twice counts once.

Limits of this design.

- Sync is one-way, from Postgres to Firestore. Edits made directly in Firestore are overwritten by the next sync.
- When Postgres is down, reads work but new forms and submissions fail.
- Syncing is manual (an endpoint you call); there is no scheduler.

4 Step-by-Step Build

Step 1: Create the Next.js project with Tailwind v4

```
npx create-next-app@latest survey-forms --ts --tailwind --eslint --app --no-src-dir --use-npm --
    import-alias "@/*"
cd survey-forms
```

Files created by the generator: `package.json`, `tsconfig.json`, `next.config.ts`, `eslint.config.mjs`, `AGENTS.md`, `README.md`, and the folders `app/` and `public/`.

No `src/` folder. Every folder in this guide (`app/`, `components/`, `lib/` (with `lib/server/` and `lib/schemas/` inside), `prisma/`, `generated/`) sits at the project root. The `@/` alias points to the root through `"paths": { "@/*": ["./*"] }` in `tsconfig.json`, which the starter writes for you.

Tailwind v4. `app/globals.css` begins with `@import "tailwindcss";`. There are no `@tailwind` directives and no `tailwind.config.js`. Create `postcss.config.mjs` (listing in Section 7) so the styling does not depend on the starter's Turbopack rule.

Cache Components. The starter sets `cacheComponents: true`. Two rules follow, and the code already applies both. A route handler must not export `runtime`. Any page that awaits `params`, and any client component that creates random ids (react-hook-form's field array does), must sit inside a `Suspense` boundary, or `next build` fails.

Step 2: Install dependencies

Run each command on one line.

```
npm i zod react-hook-form @hookform/resolvers @better-fetch/fetch hono @hono/zod-validator
    firebase-admin pg dotenv @prisma/client@7.10.0 @prisma/adapter-pg@7.10.0
npm i -D prisma@7.10.0 @types/pg prettier
npm i tailwindcss @tailwindcss/postcss postcss
```

Step 3: Configure environment variables, Firestore and Git

Files created: `.env` (never committed) and `.env.example` (committed). Next.js loads `.env`; the Prisma CLI loads it through `dotenv`. Generate the token with `openssl rand -hex 24`.

Firebase values. Copy `project_id`, `client_email` and `private_key` from the downloaded service account JSON into `FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL` and `FIREBASE_PRIVATE_KEY`. Keep the private key in double quotes on one line with the line breaks written as `\n`; `lib/server/firestore.ts` turns them back into real line breaks. Then delete the JSON file or keep it outside the project: it grants full admin access.

Create the database. In the Firebase console open Firestore and click Create database, accept the `(default)` database and choose a region. Security rules do not matter for this app, because the Admin SDK bypasses them and no browser talks to Firestore.

File updated: `.gitignore`. The starter already ignores `.env*`, which also hides the example file. Add:

```
!.env.example
/generated/prisma
*-firebase-adminsdk-*.json
serviceAccount*.json
```

Step 4: Define the database with Prisma

Files created: `prisma.config.ts` and `prisma/schema.prisma`. Prisma 7 reads the database URL from `prisma.config.ts`, writes the client to `generated/prisma`, and needs the `pg` driver adapter at runtime.

Model	What it represents	Why it is a separate model
`Form`	One survey (title, description)	Parent record; owns questions and submissions
`Question`	One prompt in a form	Needs its own type, options, order and required flag, and answers point to it
`Submission`	One respondent's attempt	Groups answers so each response can be counted, paged and deleted as a unit
`Answer`	One value for one question	Joins a submission to a question

```
npx prisma migrate dev --name init
npx prisma generate
```

Application code imports the client as `@/generated/prisma/client`. Do not drop or reset the database schema on a machine that holds real forms; a reset deletes them, and the Firestore mirror is only repopulated from Postgres, not the other way round.

Step 5: Server foundations

- `lib/server/env.ts`: validates the environment with Zod. Two lazy getters keep the database variables apart from the Firebase ones, so forms keep working when Firebase is not configured.
- `lib/server/db.ts`: one shared Prisma client with the Postgres adapter, kept on `globalThis` so hot reloading does not open new pools.
- `lib/server/firestore.ts`: starts the Firebase Admin app once (reused if it exists) and returns the Firestore database.
- `lib/server/errors.ts`: one error body shape, `{ error: { code, message } }`.
- `lib/server/methods.ts`: `methodNotAllowed`, the 405 and `Allow` helper used by every route file.
- `lib/server/validate.ts`: a wrapper around `zValidator` that returns the same 400 body with the list of issues.

Step 6: Write the schemas, one file per concern

- `lib/schemas/question.ts`: question types, the question schema and `isChoiceType`.
- `lib/schemas/form.ts`: create-form payload, form response, dashboard summary and the id parameter.
- `lib/schemas/upload.ts`: allowed file types, the 700 KB limit, the pattern a file id must match, and the upload result.
- `lib/schemas/submission.ts`: the answers payload, the page of submissions and `buildAnswerSchema`.
- `lib/schemas/builder.ts`: client-only, the one file in `lib/schemas/` the server never imports.

`z.email()` and `z.iso.date()` are Zod 4 forms; with an older Zod use `z.string().email()` and `z.string().date()`.

Step 7: Build the repository layer

- `lib/server/repo/postgres/forms.ts` and `submissions.ts`: all Prisma calls. Forms are parsed with `formSchema` so bad rows are caught on the server. `createSubmission` writes the submission and its answers in one nested create (a single transaction), skips empty answers, stores numbers as numbers, and returns what it stored so the same row can be mirrored.
- `lib/server/repo/firestore/forms.ts` and `submissions.ts`: mirror writes, mirror reads and the counter transaction described in Section 3.
- `lib/server/repo/dual.ts`: `mirrorSafely` (log, do not fail) and `readWithFallback`.
- `lib/server/repo/forms.ts` and `submissions.ts`: the facade the routes import. Postgres first, then mirror; Postgres read first, then fallback.
- `lib/server/repo/files.ts`: save, load and existence-check for uploaded files.
- `lib/server/repo/sync.ts`: `getSyncStatus` and `syncAll`.

Step 8: Build the Hono API

Method	Path	Input	Result
GET	`/api/forms`	none	200, forms with counts
POST	`/api/forms`	create-form payload	201, the form; 400
GET	`/api/forms/:id`	none	200; 404
DELETE	`/api/forms/:id`	none	204; 404
POST	`/api/forms/:id/submissions`	answers, checked against that form's questions	201, `{ id }`; 400; 404
GET	`/api/forms/:id/submissions`	`?page&limit`	200, one page; file answers are download paths
POST	`/api/files`	multipart field `file`	201, `{ id }`; 400
GET	`/api/files/:id`	none	200, the file as a download; 404
GET	`/api/sync/status`	bearer token	200, comparison of both databases; 401
POST	`/api/sync`	bearer token	200, what was copied; 401

405 for every other method. Build this once, then reuse it on every path.

1. Add a small helper in `lib/server/methods.ts`, `methodNotAllowed(app, path, allowedMethods)`. It builds the `Allow` value from the allowed methods plus `OPTIONS`, registers an `OPTIONS` handler that answers 204 with that header, and registers `app.all(path, ...)` that answers 405 with the `Allow` header and the usual error body, code `method_not_allowed`.
2. Call it at the end of each route file, after the real handlers, because Hono runs handlers in the order they were registered. Pass the methods that path really supports, for example `["GET", "POST"]` for `/api/forms`. `HEAD` is answered through `GET` automatically.

3. In `lib/server/app.ts` add `app.notFound(...)` for a JSON 404 and `app.onError(...)` for a generic 500 that logs the real error and never sends it to the client.
4. Check it: `curl -i -X DELETE /api/forms` should return 405 with `Allow: GET, POST, OPTIONS`, and an unknown path should return the JSON 404.

Error bodies look like `{ error: { code, message, issues? } }`.

Submissions. The route builds the answer schema from the form's own questions on every request, validates the body with it, checks that every file answer is the id of a file this app stored, and only then saves. A body that is not JSON becomes `null` and fails validation with a 400.

Files. The upload route re-checks type and size on the server, because the browser check can be skipped. Downloads are sent as attachments with `X-Content-Type-Options: nosniff`, so an uploaded file is never run as a page on this site.

Sync routes. They can rewrite a whole database and reveal counts, so they answer 401 unless the request carries `Authorization: Bearer <SYNC_TOKEN>`. With no token configured they stay closed.

Step 9: Mount Hono inside Next.js

`app/api/[[...route]]/route.ts` exports GET, POST, PUT, PATCH, DELETE and OPTIONS, all handled by the Hono app. All methods are exported on purpose: Next.js answers an unexported method with its own empty 405, while Hono answers with the JSON 405 and the `Allow` header. There is no `runtime` export, because Cache Components rejects it.

Step 10: Create the typed HTTP client and class patterns

`lib/fetch.ts` lists every JSON endpoint with its schemas, so a mistyped URL or wrong body is a type error and every response is validated. `getErrorMessage` reads the server's error message. File upload uses a plain `fetch` with `FormData` because a multipart body cannot be described by a Zod input schema; its response is still parsed with a schema. `lib/useLoad.ts` runs an async loader and reports loading, error or data. `lib/responsive/pattrens/forms.ts` holds every Tailwind class string, one object per component; components import their object as `p` and write `className={p.input}`. The folder keeps the spelling `pattrens` used in your other projects.

Step 11: Build the frontend

File	Responsibility
`app/page.tsx`	Dashboard: "New form" button and the list of forms.
`app/forms/new/page.tsx`	Builder page; the builder sits inside `Suspense`.
`app/f/[formId]/page.tsx`	Public survey page; params are read inside `Suspense`.
`app/forms/[id]/responses/page.tsx`	Responses page; params are read inside `Suspense`.
`app/layout.tsx`	Root layout. `<body>` has `suppressHydrationWarning` because browser extensions such as Grammarly add attributes before React loads.
`components/FormBuilder.tsx`	Create a form: react-hook-form with a field array and `builderFormSchema`.
`components/OptionsEditor.tsx`	Choices of one question: one input per option, add and remove, and a Yes / No preset.
`components/QuestionField.tsx`	One input per question type.
`components/FileUploadField.tsx`	Checks the file, uploads it to `/api/files`, stores the returned id.
`components/SurveyForm.tsx`	Fills in a survey with a schema built from its questions, then submits.
`components/SurveyLoader.tsx`	Loads a form for the public page.
`components/FormList.tsx`	Dashboard list with links to the survey and its responses.
`components/ResponsesView.tsx`	Table with one column per question and paging; file answers are links.

Step 12: Verify

```
npx prettier --check .
npx tsc --noEmit
npx eslint .
npm run build
npm run dev
```

Add a `.prettierignore` listing `.next`, `.agents`, `.claude`, `generated`, `node_modules` and `package-lock.json` so Prettier skips generated files. Do not run `npm run build` while `npm run dev` is serving the same folder: the build replaces the `.next` directory and can stop the dev server.

Then test by hand: create a form with one question of each type, open its public link, submit an invalid answer (an error appears next to that question), submit a valid one, and open the responses page.

Step 13: Operate the two databases

```
curl -H "Authorization: Bearer $SYNC_TOKEN" localhost:3000/api/sync/status
curl -X POST -H "Authorization: Bearer $SYNC_TOKEN" localhost:3000/api/sync
```

`status` lists form ids missing from Firestore, ids that exist only in Firestore, and both submission counts; `inSync` is true only when all of these match. Comparing ids matters: counts alone say "in sync" when one form was deleted and another added. `POST /api/sync` copies every form and

submission from Postgres (documents are replaced, so repeating it is safe), sets each form's counter to the real number, and removes Firestore forms that no longer exist in Postgres. Run it after any Firestore outage, after restoring Postgres from a backup, or when the server log shows "Firestore mirror failed".

5 What Was Verified

Checked in this project:

- `prettier -check`, `tsc -noEmit`, `eslint .` and `npm run build` all pass.
- API, against a running dev server with a real Postgres (Docker) and a real Firestore database: creating forms; rejecting a choice question with one option; rejecting invalid answers with one message per field; saving a valid submission and listing it with numbers stored as numbers; dashboard counts; 404 and 405 bodies.
- Files: upload of a text file, download as an attachment with `nosniff`, rejection of a disallowed type and of an 800 KB file, rejection of a malformed and of an unknown file id on submit, acceptance of a real id.
- Sync: new writes appear in both databases with equal counts; a form deleted from Firestore and a stray form added were both reported and repaired; deleting a form removes it from both; with the Postgres container stopped, the form list, a form and its submissions were still served from Firestore, and the databases matched again after Postgres returned.
- Browser: the builder creates a multi-choice form using the Yes / No preset; the public survey page renders the question and options; the hydration warning caused by a browser extension no longer shows.

Not checked:

- A full click-through of every question type in the browser (date and number inputs, the file input and the validation messages shown next to fields). They were checked through the API and the shared schemas only.
- The "Firestore mirror failed" path with a real Firestore error. Drift was created by editing Firestore by hand, not by a failing write.
- Deployment, concurrent writers, large data volumes, and a Firebase Storage bucket (not used).

6 Known Limits

- **No login.** Anyone with a form link can answer it, and the forms list, responses and file downloads need no sign-in. File ids are random and unguessable, but the responses endpoint returns them.
- **File size.** Files are limited to 700 KB because they are stored inside Firestore documents. Photos and short PDFs fit; larger files need a real bucket.
- **Orphaned files.** Deleting a form does not delete the files its answers pointed to.
- **One-way sync and manual repair**, as described in Section 3.
- **No Delete button.** Forms can only be deleted through the API.

7 Source Code

Every listing below is the complete content of the file, so you can type it in step by step. The set was type-checked together (`tsc -noEmit`) and formatted with Prettier. The code follows these conventions: braces on every `if`, a doc comment on each function, descriptive names, Prettier width 80 with single quotes and trailing commas, and no inline Tailwind classes (they live in the patterns file). Comments marked "Why" explain a decision, not the syntax.

7.1 Configuration

File: `prisma.config.ts`. Prisma 7 configuration: schema path, migrations path and database URL.

```
import 'dotenv/config';
import { defineConfig } from 'prisma/config';

/** Prisma 7 reads the schema path, migrations path and database URL here. */
export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: { path: 'prisma/migrations' },
  datasource: { url: process.env['DATABASE_URL'] },
});
```

File: `prisma/schema.prisma`. The four models, with a comment on why each exists.

```
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}

datasource db {
  provider = "postgresql"
}

// Why: the parent record. It owns the questions and the submissions.
model Form {
  id          String       @id @default(cuid())
  title       String
  description String?
  createdAt   DateTime     @default(now())
  questions   Question[]
  submissions Submission[]
}

// Why: a question needs its own type, options, order and required flag,
// and answers point to it.
model Question {
  id       String   @id @default(cuid())
  formId   String
  form     Form     @relation(fields: [formId], references: [id], onDelete: Cascade)
  label    String
  type     String
  required Boolean  @default(false)
  options  String[]
  position Int
  answers  Answer[]
}

// Why: groups answers so one response can be counted, paged and deleted
// as a unit.
model Submission {
```

```
  id        String   @id @default(cuid())
  formId    String
  form      Form     @relation(fields: [formId], references: [id], onDelete: Cascade)
  createdAt DateTime @default(now())
  answers   Answer[]
}

// Why: joins a submission to a question. A number is stored in `number`,
// everything else (including a file id) in `text`.
model Answer {
  id           String     @id @default(cuid())
  submissionId String
  submission   Submission @relation(fields: [submissionId], references: [id], onDelete: Cascade)
  questionId   String
  question     Question   @relation(fields: [questionId], references: [id], onDelete: Cascade)
  text         String?
  number       Float?
}
```

File: `postcss.config.mjs`. Tailwind v4 PostCSS plugin.

```
/** Tailwind v4 runs as a PostCSS plugin, independent of the bundler. */
const config = {
  plugins: {
    '@tailwindcss/postcss': {},
  },
};

export default config;
```

File: `.env.example`. Environment variable template.

```
# PostgreSQL connection string
DATABASE_URL=postgresql://user:pass@localhost:5432/surveys

# Firebase Admin SDK service account (Project settings, Service accounts)
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
# One line, in double quotes, with line breaks written as \n
FIREBASE_PRIVATE_KEY=

# Bearer token for GET /api/sync/status and POST /api/sync
SYNC_TOKEN=
```

File: `.prettierrc`. Prettier settings: width 80, single quotes, trailing commas.

```
{
  "printWidth": 80,
  "singleQuote": true,
  "trailingComma": "all"
}
```

7.2 Shared API schemas

File: `lib/schemas/question.ts`. Question types, schema and helper.

```
import { z } from 'zod';

export const questionTypes = [
  'short_text',
  'long_text',
```

```
  'number',
  'email',
  'date',
  'single_choice',
  'multi_choice',
  'file',
] as const;

export const questionTypeSchema = z.enum(questionTypes);
export type QuestionType = z.infer<typeof questionTypeSchema>;

/** True for the types that need a list of options. */
export function isChoiceType(type: QuestionType): boolean {
  return type === 'single_choice' || type === 'multi_choice';
}

const questionBase = z.object({
  label: z.string().trim().min(1, 'Question text is required').max(200),
  type: questionTypeSchema,
  required: z.boolean(),
  options: z.array(z.string().trim().min(1).max(100)).max(20).default([]),
});

/** A choice question needs at least two options. */
function requireOptionsForChoice(
  question: { type: QuestionType; options: string[] },
  ctx: z.RefinementCtx,
): void {
  if (isChoiceType(question.type) && question.options.length < 2) {
    ctx.addIssue({
      code: 'custom',
      path: ['options'],
      message: 'A choice question needs at least two options',
    });
  }
}

/** A question as sent when creating a form. */
export const questionInputSchema = questionBase.superRefine(
  requireOptionsForChoice,
);

/** A question as stored and returned: it also has an id and a position. */
export const questionSchema = questionBase
  .extend({ id: z.string().min(1), position: z.number().int().min(0) })
  .superRefine(requireOptionsForChoice);

export type QuestionInput = z.infer<typeof questionInputSchema>;
export type Question = z.infer<typeof questionSchema>;
```

File: `lib/schemas/form.ts`. Create-form payload, form response, dashboard summary.

```
import { z } from 'zod';
import { questionInputSchema, questionSchema } from './question';

export const createFormSchema = z.object({
  title: z.string().trim().min(1, 'Title is required').max(200),
  description: z.string().trim().max(2000).optional(),
  questions: z
    .array(questionInputSchema)
    .min(1, 'Add at least one question')
    .max(50),
});
```

```
export const formSchema = z.object({
  id: z.string().min(1),
  title: z.string(),
  description: z.string().nullable(),
  createdAt: z.string(),
  questions: z.array(questionSchema),
});

/** One row of the dashboard list. */
export const formSummarySchema = z.object({
  id: z.string().min(1),
  title: z.string(),
  createdAt: z.string(),
  questionCount: z.number().int(),
  responseCount: z.number().int(),
});

export const formIdParamSchema = z.object({ id: z.string().min(1) });

export type CreateFormInput = z.infer<typeof createFormSchema>;
export type Form = z.infer<typeof formSchema>;
export type FormSummary = z.infer<typeof formSummarySchema>;
```

File: `lib/schemas/upload.ts`. Upload limits, allowed types, file id pattern.

```
import { z } from 'zod';

/** Files are stored inside Firestore documents, which are capped at 1 MiB. */
export const MAX_FILE_BYTES = 700 * 1024;

export const ALLOWED_FILE_TYPES = [
  'application/pdf',
  'image/png',
  'image/jpeg',
  'text/plain',
] as const;

/** A file id is 32 lowercase hex characters (a UUID without dashes). */
export const FILE_ID_PATTERN = /^[a-f0-9]{32}$/;

export const fileIdSchema = z
  .string()
  .regex(FILE_ID_PATTERN, 'Invalid file id');
export const fileIdParamSchema = z.object({ id: fileIdSchema });
export const uploadResultSchema = z.object({ id: fileIdSchema });

export type UploadResult = z.infer<typeof uploadResultSchema>;
```

File: `lib/schemas/submission.ts`. Answers, submissions page, and the runtime answer schema.

```
import { z } from 'zod';
import { fileIdSchema } from './upload';
import { isChoiceType, type Question } from './question';

/** One answer is text, or a list of texts for a multi-choice question. */
const answerValueSchema = z.union([z.string(), z.array(z.string())]);

export const answersPayloadSchema = z.object({
  answers: z.record(z.string(), answerValueSchema),
});

export const submissionSchema = z.object({
  id: z.string().min(1),
  createdAt: z.string(),
```

```
  answers: z.record(z.string(), answerValueSchema),
});

export const pageQuerySchema = z.object({
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20),
});

export const submissionsPageSchema = z.object({
  items: z.array(submissionSchema),
  page: z.number().int(),
  limit: z.number().int(),
  total: z.number().int(),
});

export const submissionIdSchema = z.object({ id: z.string().min(1) });

export type Submission = z.infer<typeof submissionSchema>;
export type SubmissionsPage = z.infer<typeof submissionsPageSchema>;

/** Builds the rule for one question's answer. */
function answerSchemaFor(question: Question): z.ZodType<unknown> {
  let schema: z.ZodType<unknown>;
  switch (question.type) {
    case 'number':
      schema = z.string().refine((v) => v === '' || !Number.isNaN(Number(v)), {
        message: 'Enter a number',
      });
      break;
    case 'email':
      schema = z.union([z.literal(''), z.email('Enter a valid email')]);
      break;
    case 'date':
      schema = z.union([z.literal(''), z.iso.date('Enter a valid date')]);
      break;
    case 'file':
      schema = z.union([z.literal(''), fileIdSchema]);
      break;
    case 'single_choice':
      schema = z.union([
        z.literal(''),
        z.enum(question.options as [string, ...string[]]),
      ]);
      break;
    case 'multi_choice':
      schema = z.array(z.enum(question.options as [string, ...string[]]));
      break;
    default:
      schema = z.string().max(5000);
  }
  if (!question.required) {
    return schema;
  }
  return schema.refine(
    (v) => (Array.isArray(v) ? v.length > 0 : v !== '' && v !== undefined),
    { message: 'This question is required' },
  );
}

/**
 * Builds the answer schema from a form's own questions, so the browser form
 * and the API apply exactly the same rules. Answers are keyed by question id.
 */
export function buildAnswerSchema(questions: Question[]) {
  const shape: Record<string, z.ZodType<unknown>> = {};
```

```
  for (const question of questions) {
    const base = answerSchemaFor(question);
    shape[question.id] = question.required
      ? base
      : isChoiceType(question.type) && question.type === 'multi_choice'
        ? base.default([])
        : base.optional();
  }
  return z.object(shape);
}
```

7.3 Server foundations

File: `lib/server/env.ts`. Lazy, validated environment variables.

```
import { z } from 'zod';

const databaseSchema = z.object({ DATABASE_URL: z.string().min(1) });

const firebaseSchema = z.object({
  FIREBASE_PROJECT_ID: z.string().min(1),
  FIREBASE_CLIENT_EMAIL: z.string().min(1),
  FIREBASE_PRIVATE_KEY: z.string().min(1),
});

/**
 * Why two lazy getters: each one validates only when called, so forms keep
 * working when Firebase is not configured.
 */
export function getDatabaseEnv() {
  return databaseSchema.parse(process.env);
}

export function getFirebaseEnv() {
  return firebaseSchema.parse(process.env);
}

/** The sync token, or undefined when none is set (sync stays closed). */
export function getSyncToken(): string | undefined {
  return process.env.SYNC_TOKEN || undefined;
}
```

File: `lib/server/db.ts`. Shared Prisma client with the Postgres adapter.

```
import { PrismaPg } from '@prisma/adapter-pg';
import { PrismaClient } from '@/generated/prisma/client';
import { getDatabaseEnv } from './env';

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };

/** Creates the Prisma client with the node-postgres adapter. */
function createClient(): PrismaClient {
  const adapter = new PrismaPg({
    connectionString: getDatabaseEnv().DATABASE_URL,
  });
  return new PrismaClient({ adapter });
}

/**
 * Why globalThis: hot reloading re-runs this module, and a new client each
 * time would open a new connection pool.
 */
```

```
export const prisma = globalForPrisma.prisma ?? createClient();

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma;
}
```

File: `lib/server/firestore.ts`. Firebase Admin app and Firestore database.

```
import { cert, getApps, initializeApp } from 'firebase-admin/app';
import { getFirestore } from 'firebase-admin/firestore';
import { getFirebaseEnv } from './env';

/** Starts the Firebase Admin app once and returns the Firestore database. */
export function getDb() {
  const env = getFirebaseEnv();
  const app =
    getApps()[0] ??
    initializeApp({
      credential: cert({
        projectId: env.FIREBASE_PROJECT_ID,
        clientEmail: env.FIREBASE_CLIENT_EMAIL,
        // The key is stored on one line with \n; turn it back into line breaks.
        privateKey: env.FIREBASE_PRIVATE_KEY.replace(/\\n/g, '\n'),
      }),
    });
  return getFirestore(app);
}
```

File: `lib/server/errors.ts`. The one error body shape.

```
import type { Context } from 'hono';
import type { ContentfulStatusCode } from 'hono/utils/http-status';

export type ApiErrorBody = {
  error: { code: string; message: string; issues?: unknown };
};

/** The one error body shape used by every endpoint. */
export function errorBody(
  code: string,
  message: string,
  issues?: unknown,
): ApiErrorBody {
  return {
    error: { code, message, ...(issues === undefined ? {} : { issues }) },
  };
}

/** Sends the error body with a status and optional headers. */
export function jsonError(
  c: Context,
  status: ContentfulStatusCode,
  code: string,
  message: string,
  headers?: Record<string, string>,
) {
  return c.json(errorBody(code, message), status, headers);
}
```

File: `lib/server/methods.ts`. Helper that adds the 405 + Allow catch-all to a path.

```
import type { Hono } from 'hono';
import { jsonError } from './errors';
```

```
/**
 * Answers every method not handled on `path` with a JSON 405 and an `Allow`
 * header. Call it AFTER the real handlers for the path, because Hono runs
 * handlers in the order they were registered.
 */
export function methodNotAllowed(app: Hono, path: string, allowed: string[]) {
  const allow = [...new Set([...allowed, 'OPTIONS'])].join(', ');
  app.options(path, (c) => c.body(null, 204, { Allow: allow }));
  app.all(path, (c) =>
    jsonError(
      c,
      405,
      'method_not_allowed',
      `${c.req.method} is not allowed here. Allowed: ${allow}.`,
      { Allow: allow },
    ),
  );
}
```

File: `lib/server/validate.ts`. zValidator wrapper with a consistent 400 body.

```
import { zValidator } from '@hono/zod-validator';
import type { ZodType } from 'zod';
import { errorBody } from './errors';

type Target = 'json' | 'param' | 'query';

/** zValidator with the same 400 body everywhere, listing every issue. */
export function validate<T extends ZodType>(target: Target, schema: T) {
  return zValidator(target, schema, (result, c) => {
    if (!result.success) {
      const issues = result.error.issues.map((issue) => ({
        path: issue.path.join('.'),
        message: issue.message,
      }));
      return c.json(
        errorBody('invalid_request', 'Invalid request', issues),
        400,
      );
    }
  });
}
```

7.4 Repository layer

File: `lib/server/repo/postgres/forms.ts`. Form data access in Postgres (source of truth).

```
import { prisma } from '../../db';
import {
  formSchema,
  type CreateFormInput,
  type Form,
  type FormSummary,
} from '@/lib/schemas/form';

/** Lists every form with its question and response counts. */
export async function listForms(): Promise<FormSummary[]> {
  const rows = await prisma.form.findMany({
    orderBy: { createdAt: 'desc' },
    include: { _count: { select: { questions: true, submissions: true } } },
  });
```

```
  return rows.map((row) => ({
    id: row.id,
    title: row.title,
    createdAt: row.createdAt.toISOString(),
    questionCount: row._count.questions,
    responseCount: row._count.submissions,
  }));
}

/** Returns one form with its questions in order, or null if it is missing. */
export async function getForm(id: string): Promise<Form | null> {
  const row = await prisma.form.findUnique({
    where: { id },
    include: { questions: { orderBy: { position: 'asc' } } },
  });
  if (!row) {
    return null;
  }
  // Parsing catches a bad row on the server instead of in the browser.
  return formSchema.parse({ ...row, createdAt: row.createdAt.toISOString() });
}

/** Creates a form and its questions in one nested write. */
export async function createForm(input: CreateFormInput): Promise<Form> {
  const row = await prisma.form.create({
    data: {
      title: input.title,
      description: input.description ?? null,
      questions: {
        create: input.questions.map((q, position) => ({ ...q, position })),
      },
    },
    include: { questions: { orderBy: { position: 'asc' } } },
  });
  return formSchema.parse({ ...row, createdAt: row.createdAt.toISOString() });
}

/** Deletes a form; its questions and submissions go with it. Returns false if missing. */
export async function deleteForm(id: string): Promise<boolean> {
  const result = await prisma.form.deleteMany({ where: { id } });
  return result.count > 0;
}
```

File: `lib/server/repo/postgres/submissions.ts`. Submission data access in Postgres.

```
import { prisma } from '../../db';
import type { Form } from '@/lib/schemas/form';
import type { Submission } from '@/lib/schemas/submission';

type Answers = Record<string, string | string[]>;

/** Returns what was stored so the same row can be mirrored to Firestore. */
export async function createSubmission(
  form: Form,
  answers: Answers,
): Promise<Submission> {
  const rows: {
    questionId: string;
    text: string | null;
    number: number | null;
  }[] = [];
  for (const question of form.questions) {
    const value = answers[question.id];
    if (value === undefined || value === '' || value.length === 0) {
```

```
      continue; // empty answers are not stored
    }
    if (question.type === 'number') {
      rows.push({ questionId: question.id, text: null, number: Number(value) });
    } else {
      const text = Array.isArray(value) ? JSON.stringify(value) : value;
      rows.push({ questionId: question.id, text, number: null });
    }
  }
  // One nested create is a single transaction.
  const saved = await prisma.submission.create({
    data: { formId: form.id, answers: { create: rows } },
  });
  return {
    id: saved.id,
    createdAt: saved.createdAt.toISOString(),
    answers: toAnswerMap(form, rows),
  };
}

/** Rebuilds the question-id keyed answer map from stored rows. */
function toAnswerMap(
  form: Form,
  rows: { questionId: string; text: string | null; number: number | null }[],
): Answers {
  const map: Answers = {};
  for (const row of rows) {
    const question = form.questions.find((q) => q.id === row.questionId);
    if (row.number !== null) {
      map[row.questionId] = String(row.number);
    } else if (question?.type === 'multi_choice' && row.text) {
      map[row.questionId] = JSON.parse(row.text) as string[];
    } else if (row.text !== null) {
      map[row.questionId] = row.text;
    }
  }
  return map;
}

/** Lists one page of a form's submissions, newest first. */
export async function listSubmissions(form: Form, page: number, limit: number) {
  const [total, rows] = await Promise.all([
    prisma.submission.count({ where: { formId: form.id } }),
    prisma.submission.findMany({
      where: { formId: form.id },
      orderBy: { createdAt: 'desc' },
      skip: (page - 1) * limit,
      take: limit,
      include: { answers: true },
    }),
  ]);
  const items: Submission[] = rows.map((row) => ({
    id: row.id,
    createdAt: row.createdAt.toISOString(),
    answers: toAnswerMap(form, row.answers),
  }));
  return { items, page, limit, total };
}

/** Every submission of a form (used by sync). */
export async function listAllSubmissions(form: Form): Promise<Submission[]> {
  const rows = await prisma.submission.findMany({
    where: { formId: form.id },
    include: { answers: true },
  });
```

```
  return rows.map((row) => ({
    id: row.id,
    createdAt: row.createdAt.toISOString(),
    answers: toAnswerMap(form, row.answers),
  }));
}
```

File: `lib/server/repo/firestore/forms.ts`. Form mirror writes and reads in Firestore.

```
import { getDb } from '../../firestore';
import { formSchema, type Form, type FormSummary } from '@/lib/schemas/form';

/** Copies a form to Firestore under the same id. Keeps the response counter. */
export async function mirrorForm(form: Form, responseCount?: number) {
  const ref = getDb().collection('forms').doc(form.id);
  const data: Record<string, unknown> = { ...form };
  if (responseCount !== undefined) {
    data.responseCount = responseCount;
  }
  await ref.set(data, { merge: true });
  if (responseCount === undefined) {
    // Set the counter only when the document has none yet.
    await getDb().runTransaction(async (tx) => {
      const snap = await tx.get(ref);
      if (snap.get('responseCount') === undefined) {
        tx.update(ref, { responseCount: 0 });
      }
    });
  }
}

export async function deleteMirroredForm(id: string) {
  const ref = getDb().collection('forms').doc(id);
  await getDb().recursiveDelete(ref);
}

export async function listMirroredForms(): Promise<FormSummary[]> {
  const snap = await getDb()
    .collection('forms')
    .orderBy('createdAt', 'desc')
    .get();
  return snap.docs.map((doc) => ({
    id: doc.id,
    title: doc.get('title'),
    createdAt: doc.get('createdAt'),
    questionCount: (doc.get('questions') ?? []).length,
    responseCount: doc.get('responseCount') ?? 0,
  }));
}

export async function getMirroredForm(id: string): Promise<Form | null> {
  const snap = await getDb().collection('forms').doc(id).get();
  if (!snap.exists) {
    return null;
  }
  return formSchema.parse({ ...snap.data(), id: snap.id });
}

export async function listMirroredFormIds(): Promise<string[]> {
  const snap = await getDb().collection('forms').select().get();
  return snap.docs.map((doc) => doc.id);
}
```

File: `lib/server/repo/firestore/submissions.ts`. Submission mirror writes and reads in Firestore.

```
import { FieldValue } from 'firebase-admin/firestore';
import { getDb } from '../../firestore';
import type { Submission } from '@/lib/schemas/submission';

/**
 * Writes the submission and bumps the form counter in one transaction, and
 * only if the document did not exist, so mirroring twice counts once.
 */
export async function mirrorSubmission(formId: string, submission: Submission) {
  const db = getDb();
  const formRef = db.collection('forms').doc(formId);
  const ref = formRef.collection('submissions').doc(submission.id);
  await db.runTransaction(async (tx) => {
    const existing = await tx.get(ref);
    tx.set(ref, submission);
    if (!existing.exists) {
      tx.set(
        formRef,
        { responseCount: FieldValue.increment(1) },
        { merge: true },
      );
    }
  });
}

export async function listMirroredSubmissions(
  formId: string,
  page: number,
  limit: number,
) {
  const col = getDb().collection('forms').doc(formId).collection('submissions');
  const [totalSnap, snap] = await Promise.all([
    col.count().get(),
    col
      .orderBy('createdAt', 'desc')
      .offset((page - 1) * limit)
      .limit(limit)
      .get(),
  ]);
  const items = snap.docs.map((doc) => doc.data() as Submission);
  return { items, page, limit, total: totalSnap.data().count };
}

export async function countMirroredSubmissions(
  formId: string,
): Promise<number> {
  const col = getDb().collection('forms').doc(formId).collection('submissions');
  return (await col.count().get()).data().count;
}
```

File: `lib/server/repo/dual.ts`. Safe mirror write and read with fallback.

```
/**
 * Runs a Firestore mirror write. A failure is logged and swallowed, because
 * the response is already correct; the sync endpoint repairs the difference.
 */
export async function mirrorSafely(
  task: () => Promise<unknown>,
): Promise<void> {
  try {
    await task();
```

```
  } catch (error) {
    console.error('Firestore mirror failed', error);
  }
}

/**
 * Reads from Postgres. If Postgres throws (for example it is unreachable),
 * the same read runs against Firestore. A "not found" (null) is a real
 * answer and is not retried.
 */
export async function readWithFallback<T>(
  primary: () => Promise<T>,
  fallback: () => Promise<T>,
): Promise<T> {
  try {
    return await primary();
  } catch (error) {
    console.error('Postgres read failed, using Firestore', error);
    return fallback();
  }
}
```

File: `lib/server/repo/forms.ts`. Form facade: Postgres first, then mirror.

```
import type { CreateFormInput } from '@/lib/schemas/form';
import { mirrorSafely, readWithFallback } from './dual';
import * as firestore from './firestore/forms';
import * as postgres from './postgres/forms';

export function listForms() {
  return readWithFallback(postgres.listForms, firestore.listMirroredForms);
}

export function getForm(id: string) {
  return readWithFallback(
    () => postgres.getForm(id),
    () => firestore.getMirroredForm(id),
  );
}

/** Postgres first, then the mirror. */
export async function createForm(input: CreateFormInput) {
  const form = await postgres.createForm(input);
  await mirrorSafely(() => firestore.mirrorForm(form));
  return form;
}

export async function deleteForm(id: string) {
  const deleted = await postgres.deleteForm(id);
  if (deleted) {
    await mirrorSafely(() => firestore.deleteMirroredForm(id));
  }
  return deleted;
}
```

File: `lib/server/repo/submissions.ts`. Submission facade: Postgres first, then mirror.

```
import type { Form } from '@/lib/schemas/form';
import { mirrorSafely, readWithFallback } from './dual';
import * as firestore from './firestore/submissions';
import * as postgres from './postgres/submissions';

/** Postgres first, then the mirror. */
export async function createSubmission(
```

```
  form: Form,
  answers: Record<string, string | string[]>,
) {
  const submission = await postgres.createSubmission(form, answers);
  await mirrorSafely(() => firestore.mirrorSubmission(form.id, submission));
  return submission;
}

export function listSubmissions(form: Form, page: number, limit: number) {
  return readWithFallback(
    () => postgres.listSubmissions(form, page, limit),
    () => firestore.listMirroredSubmissions(form.id, page, limit),
  );
}
```

File: `lib/server/repo/files.ts`. Uploaded files stored in Firestore.

```
import { randomUUID } from 'node:crypto';
import { getDb } from '../firestore';

type StoredFile = { name: string; type: string; data: string };

/** Saves a file as a Firestore document and returns its random id. */
export async function saveFile(file: File): Promise<string> {
  const id = randomUUID().replace(/-/g, '');
  const bytes = Buffer.from(await file.arrayBuffer());
  const doc: StoredFile = {
    name: file.name,
    type: file.type,
    data: bytes.toString('base64'),
  };
  await getDb().collection('files').doc(id).set(doc);
  return id;
}

/** Loads a stored file, or null if the id is unknown. */
export async function loadFile(id: string) {
  const snap = await getDb().collection('files').doc(id).get();
  if (!snap.exists) {
    return null;
  }
  const doc = snap.data() as StoredFile;
  return {
    name: doc.name,
    type: doc.type,
    bytes: Buffer.from(doc.data, 'base64'),
  };
}

export async function fileExists(id: string): Promise<boolean> {
  return (await getDb().collection('files').doc(id).get()).exists;
}
```

File: `lib/server/repo/sync.ts`. Sync status and full repair.

```
import { prisma } from '../db';
import * as firestoreForms from './firestore/forms';
import * as firestoreSubmissions from './firestore/submissions';
import * as postgresForms from './postgres/forms';
import * as postgresSubmissions from './postgres/submissions';

/**
 * Compares both databases by id, not only by count: counts alone say
 * "in sync" when one form was deleted and another added.
```

```
 */
export async function getSyncStatus() {
  const [pgIds, fsIds] = await Promise.all([
    prisma.form
      .findMany({ select: { id: true } })
      .then((r) => r.map((f) => f.id)),
    firestoreForms.listMirroredFormIds(),
  ]);
  const missingInFirestore = pgIds.filter((id) => !fsIds.includes(id));
  const onlyInFirestore = fsIds.filter((id) => !pgIds.includes(id));
  const pgSubmissions = await prisma.submission.count();
  let fsSubmissions = 0;
  for (const id of fsIds) {
    fsSubmissions += await firestoreSubmissions.countMirroredSubmissions(id);
  }
  return {
    missingInFirestore,
    onlyInFirestore,
    submissions: { postgres: pgSubmissions, firestore: fsSubmissions },
    inSync:
      missingInFirestore.length === 0 &&
      onlyInFirestore.length === 0 &&
      pgSubmissions === fsSubmissions,
  };
}

/** Copies everything from Postgres to Firestore. Safe to repeat. */
export async function syncAll() {
  const summaries = await postgresForms.listForms();
  let submissionCount = 0;
  for (const summary of summaries) {
    const form = await postgresForms.getForm(summary.id);
    if (!form) {
      continue;
    }
    const submissions = await postgresSubmissions.listAllSubmissions(form);
    for (const submission of submissions) {
      await firestoreSubmissions.mirrorSubmission(form.id, submission);
    }
    // Why last: it sets the counter to the real number, whatever the
    // submission mirrors incremented along the way.
    await firestoreForms.mirrorForm(form, submissions.length);
    submissionCount += submissions.length;
  }
  const keep = new Set(summaries.map((s) => s.id));
  const stray = (await firestoreForms.listMirroredFormIds()).filter(
    (id) => !keep.has(id),
  );
  for (const id of stray) {
    await firestoreForms.deleteMirroredForm(id);
  }
  return {
    forms: summaries.length,
    submissions: submissionCount,
    removed: stray.length,
  };
}
```

7.5 API routes

File: `lib/server/routes/forms.ts`. Form endpoints and 405 handling.

```
import { Hono } from 'hono';
```

```
import { createFormSchema, formIdParamSchema } from '@/lib/schemas/form';
import { jsonError } from '../errors';
import { methodNotAllowed } from '../methods';
import { createForm, deleteForm, getForm, listForms } from '../repo/forms';
import { validate } from '../validate';

export const forms = new Hono();

forms.get('/', async (c) => c.json(await listForms()));

forms.post('/', validate('json', createFormSchema), async (c) => {
  const form = await createForm(c.req.valid('json'));
  return c.json(form, 201);
});
methodNotAllowed(forms, '/', ['GET', 'POST']);

forms.get('/:id', validate('param', formIdParamSchema), async (c) => {
  const form = await getForm(c.req.valid('param').id);
  return form ? c.json(form) : jsonError(c, 404, 'not_found', 'Form not found');
});

forms.delete('/:id', validate('param', formIdParamSchema), async (c) => {
  const deleted = await deleteForm(c.req.valid('param').id);
  return deleted
    ? c.body(null, 204)
    : jsonError(c, 404, 'not_found', 'Form not found');
});
methodNotAllowed(forms, '/:id', ['GET', 'DELETE']);
```

File: `lib/server/routes/submissions.ts`. Submission endpoints and 405 handling.

```
import { Hono } from 'hono';
import { formIdParamSchema } from '@/lib/schemas/form';
import {
  answersPayloadSchema,
  buildAnswerSchema,
  pageQuerySchema,
} from '@/lib/schemas/submission';
import { errorBody, jsonError } from '../errors';
import { methodNotAllowed } from '../methods';
import { getForm } from '../repo/forms';
import { fileExists } from '../repo/files';
import { createSubmission, listSubmissions } from '../repo/submissions';
import { validate } from '../validate';

export const submissions = new Hono();

submissions.post(
  '/:id/submissions',
  validate('param', formIdParamSchema),
  async (c) => {
    const form = await getForm(c.req.valid('param').id);
    if (!form) {
      return jsonError(c, 404, 'not_found', 'Form not found');
    }
    // A body that is not JSON becomes null and fails validation with a 400.
    const body = await c.req.json().catch(() => null);
    const shape = answersPayloadSchema.safeParse(body);
    if (!shape.success) {
      return c.json(errorBody('invalid_request', 'Invalid request'), 400);
    }
    // Why per request: the rules come from this form's own questions.
    const result = buildAnswerSchema(form.questions).safeParse(
      shape.data.answers,
```

```
    );
    if (!result.success) {
      const issues = result.error.issues.map((i) => ({
        path: i.path.join('.'),
        message: i.message,
      }));
      return c.json(
        errorBody('invalid_answers', 'Some answers are invalid', issues),
        400,
      );
    }
    const answers = result.data as Record<string, string | string[]>;
    // Every file answer must be the id of a file this app stored.
    for (const question of form.questions) {
      const value = answers[question.id];
      if (
        question.type === 'file' &&
        typeof value === 'string' &&
        value !== ''
      ) {
        if (!(await fileExists(value))) {
          return c.json(
            errorBody('invalid_answers', 'Unknown file', [
              { path: question.id, message: 'Unknown file' },
            ]),
            400,
          );
        }
      }
    }
    const saved = await createSubmission(form, answers);
    return c.json({ id: saved.id }, 201);
  },
);

submissions.get(
  '/:id/submissions',
  validate('param', formIdParamSchema),
  validate('query', pageQuerySchema),
  async (c) => {
    const form = await getForm(c.req.valid('param').id);
    if (!form) {
      return jsonError(c, 404, 'not_found', 'Form not found');
    }
    const { page, limit } = c.req.valid('query');
    const result = await listSubmissions(form, page, limit);
    // File answers become download paths.
    const fileIds = new Set(
      form.questions.filter((q) => q.type === 'file').map((q) => q.id),
    );
    const items = result.items.map((item) => ({
      ...item,
      answers: Object.fromEntries(
        Object.entries(item.answers).map(([key, value]) => [
          key,
          fileIds.has(key) && typeof value === 'string' && value
            ? `/api/files/${value}`
            : value,
        ]),
      ),
    }));
    return c.json({ ...result, items });
  },
);
methodNotAllowed(submissions, '/:id/submissions', ['GET', 'POST']);
```

File: `lib/server/routes/files.ts`. File upload and download.

```
import { Hono } from 'hono';
import {
  ALLOWED_FILE_TYPES,
  MAX_FILE_BYTES,
  fileIdParamSchema,
} from '@/lib/schemas/upload';
import { errorBody, jsonError } from '../errors';
import { methodNotAllowed } from '../methods';
import { loadFile, saveFile } from '../repo/files';
import { validate } from '../validate';

export const files = new Hono();

files.post('/', async (c) => {
  const form = await c.req.formData().catch(() => null);
  const file = form?.get('file');
  if (!(file instanceof File)) {
    return c.json(
      errorBody('invalid_request', 'Send a multipart field "file"'),
      400,
    );
  }
  // Re-checked here because the browser check can be skipped.
  if (!(ALLOWED_FILE_TYPES as readonly string[]).includes(file.type)) {
    return c.json(
      errorBody('invalid_file', 'This file type is not allowed'),
      400,
    );
  }
  if (file.size > MAX_FILE_BYTES) {
    return c.json(
      errorBody('invalid_file', 'The file is larger than 700 KB'),
      400,
    );
  }
  return c.json({ id: await saveFile(file) }, 201);
});
methodNotAllowed(files, '/', ['POST']);

files.get('/:id', validate('param', fileIdParamSchema), async (c) => {
  const stored = await loadFile(c.req.valid('param').id);
  if (!stored) {
    return jsonError(c, 404, 'not_found', 'File not found');
  }
  // An attachment plus nosniff, so an upload is never run as a page here.
  return new Response(new Uint8Array(stored.bytes), {
    headers: {
      'Content-Type': stored.type,
      'Content-Disposition': `attachment; filename="${encodeURIComponent(stored.name)}"`,
      'X-Content-Type-Options': 'nosniff',
    },
  });
});
methodNotAllowed(files, '/:id', ['GET']);
```

File: `lib/server/routes/sync.ts`. Token-protected sync endpoints.

```
import { Hono } from 'hono';
import { getSyncToken } from '../env';
import { jsonError } from '../errors';
import { methodNotAllowed } from '../methods';
import { getSyncStatus, syncAll } from '../repo/sync';
```

```
export const sync = new Hono();

// Why: these routes can rewrite a whole database and reveal counts. With no
// token configured they stay closed.
sync.use('*', async (c, next) => {
  const token = getSyncToken();
  if (!token || c.req.header('Authorization') !== `Bearer ${token}`) {
    return jsonError(c, 401, 'unauthorized', 'Missing or wrong token');
  }
  await next();
});

sync.get('/status', async (c) => c.json(await getSyncStatus()));
methodNotAllowed(sync, '/status', ['GET']);

sync.post('/', async (c) => c.json(await syncAll()));
methodNotAllowed(sync, '/', ['POST']);
```

File: `lib/server/app.ts`. The Hono app: routes, 404 and error handler.

```
import { Hono } from 'hono';
import { jsonError } from './errors';
import { files } from './routes/files';
import { forms } from './routes/forms';
import { submissions } from './routes/submissions';
import { sync } from './routes/sync';

export const app = new Hono().basePath('/api');

app.route('/forms', forms);
app.route('/forms', submissions);
app.route('/files', files);
app.route('/sync', sync);

// An unknown path returns a JSON 404.
app.notFound((c) => jsonError(c, 404, 'not_found', 'No such route'));

// An unexpected failure returns a generic 500; the real error is only logged.
app.onError((error, c) => {
  console.error(error);
  return jsonError(c, 500, 'internal_error', 'Something went wrong');
});
```

File: `app/api/[[...route]]/route.ts`. Mounts Hono inside Next.js.

```
import { handle } from 'hono/vercel';
import { app } from '@/lib/server/app';

// Every method is exported on purpose: Next.js answers an unexported method
// with its own empty 405, while Hono answers with the JSON 405 and Allow.
// There is no `runtime` export, because Cache Components rejects it.
export const GET = handle(app);
export const POST = handle(app);
export const PUT = handle(app);
export const PATCH = handle(app);
export const DELETE = handle(app);
export const OPTIONS = handle(app);
```

7.6 Client library

File: `lib/fetch.ts`. Typed HTTP client and error message helper.

```
import {
  BetterFetchError,
  createFetch,
  createSchema,
} from '@better-fetch/fetch';
import { z } from 'zod';
import {
  createFormSchema,
  formIdParamSchema,
  formSchema,
  formSummarySchema,
} from '@/lib/schemas/form';
import {
  answersPayloadSchema,
  pageQuerySchema,
  submissionsPageSchema,
} from '@/lib/schemas/submission';
import {
  uploadResultSchema,
  MAX_FILE_BYTES,
  ALLOWED_FILE_TYPES,
} from '@/lib/schemas/upload';

const idResult = z.object({ id: z.string() });

/** Every JSON endpoint with its schemas: a wrong URL or body is a type error. */
const schema = createSchema({
  '/forms': { output: z.array(formSummarySchema) },
  '@post/forms': { input: createFormSchema, output: formSchema },
  '/forms/:id': { params: formIdParamSchema, output: formSchema },
  '@delete/forms/:id': { params: formIdParamSchema },
  '/forms/:id/submissions': {
    params: formIdParamSchema,
    query: pageQuerySchema.partial(),
    output: submissionsPageSchema,
  },
  '@post/forms/:id/submissions': {
    params: formIdParamSchema,
    input: answersPayloadSchema,
    output: idResult,
  },
});

/** better-fetch needs an absolute base URL; requests only run in the browser. */
function apiBase(): string {
  const origin =
    typeof window === 'undefined'
      ? 'http://localhost:3000'
      : window.location.origin;
  return `${origin}/api`;
}

export const api = createFetch({ baseURL: apiBase(), schema, throw: true });

/** Reads the server's error message, or falls back to a generic one. */
export function getErrorMessage(error: unknown): string {
  if (error instanceof BetterFetchError) {
    const body = error.error as { error?: { message?: string } } | undefined;
    return body?.error?.message ?? error.statusText ?? 'Request failed';
  }
  return error instanceof Error ? error.message : 'Something went wrong';
```

```
}

/**
 * Plain fetch with FormData: a multipart body cannot be described by a Zod
 * input schema. The response is still parsed with a schema.
 */
export async function uploadFile(file: File): Promise<string> {
  if (!(ALLOWED_FILE_TYPES as readonly string[]).includes(file.type)) {
    throw new Error('This file type is not allowed');
  }
  if (file.size > MAX_FILE_BYTES) {
    throw new Error('The file is larger than 700 KB');
  }
  const body = new FormData();
  body.append('file', file);
  const response = await fetch(`${apiBase()}/files`, { method: 'POST', body });
  const json: unknown = await response.json();
  if (!response.ok) {
    throw new BetterFetchError(response.status, response.statusText, json);
  }
  return uploadResultSchema.parse(json).id;
}
```

File: `lib/useLoad.ts`. Loading hook.

```
'use client';

import { useCallback, useEffect, useState } from 'react';
import { getErrorMessage } from './fetch';

type LoadState<T> = {
  data: T | null;
  error: string | null;
  loading: boolean;
  reload: () => void;
};

/** Runs an async loader and reports loading, error or data. */
export function useLoad<T>(
  loader: () => Promise<T>,
  deps: unknown[],
): LoadState<T> {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const [version, setVersion] = useState(0);

  // eslint-disable-next-line react-hooks/exhaustive-deps
  const run = useCallback(loader, deps);

  useEffect(() => {
    let cancelled = false;
    run()
      .then((value) => {
        if (!cancelled) {
          setData(value);
          setError(null);
          setLoading(false);
        }
      })
      .catch((e: unknown) => {
        if (!cancelled) {
          setError(getErrorMessage(e));
          setLoading(false);
```

```
        }
      });
    return () => {
      cancelled = true;
    };
  }, [run, version]);

  return { data, error, loading, reload: () => setVersion((v) => v + 1) };
}
```

File: `lib/schemas/builder.ts`. Builder-only schema and conversion to the API payload.

```
import { z } from 'zod';
import { createFormSchema, type CreateFormInput } from './form';
import { isChoiceType, questionTypeSchema } from './question';

/**
 * Client-only. react-hook-form field arrays need objects, so options are
 * `{ value }` here and become plain strings in the API payload.
 */
export const builderFormSchema = z.object({
  title: z.string().trim().min(1, 'Title is required'),
  description: z.string().optional(),
  questions: z
    .array(
      z.object({
        label: z.string().trim().min(1, 'Question text is required'),
        type: questionTypeSchema,
        required: z.boolean(),
        options: z.array(z.object({ value: z.string() })),
      }),
    )
    .min(1, 'Add at least one question'),
});

export type BuilderForm = z.infer<typeof builderFormSchema>;

/** Converts the builder state to the payload the API accepts. */
export function toCreateFormPayload(values: BuilderForm): CreateFormInput {
  return createFormSchema.parse({
    title: values.title,
    description: values.description || undefined,
    questions: values.questions.map((q) => ({
      label: q.label,
      type: q.type,
      required: q.required,
      options: isChoiceType(q.type)
        ? q.options.map((o) => o.value).filter((v) => v.trim() !== '')
        : [],
    })),
  });
}
```

File: `lib/responsive/pattrens/forms.ts`. All Tailwind class strings.

```
/** Every Tailwind class string, one object per component. */

export const page = {
  main: 'mx-auto w-full max-w-3xl px-4 py-8',
  heading: 'mb-6 text-2xl font-semibold',
  muted: 'text-sm text-zinc-500',
  error: 'text-sm text-red-600',
};
```

```
export const shared = {
  input:
    'w-full rounded border border-zinc-300 px-3 py-2 text-sm focus:border-blue-500 focus:outline-
    none',
  label: 'mb-1 block text-sm font-medium',
  button:
    'rounded bg-blue-600 px-4 py-2 text-sm font-medium text-white hover:bg-blue-700 disabled:
    opacity-50',
  buttonGhost:
    'rounded border border-zinc-300 px-3 py-1.5 text-sm hover:bg-zinc-100',
  link: 'text-blue-600 hover:underline',
};

export const builder = {
  form: 'space-y-6',
  card: 'space-y-3 rounded border border-zinc-200 p-4',
  row: 'flex items-center gap-2',
  checkbox: 'h-4 w-4',
};

export const survey = {
  form: 'space-y-5',
  field: 'space-y-1',
  options: 'space-y-1',
  option: 'flex items-center gap-2 text-sm',
};

export const list = {
  header: 'mb-6 flex items-center justify-between',
  item: 'flex items-center justify-between border-b border-zinc-200 py-3',
  actions: 'flex gap-4 text-sm',
};

export const table = {
  wrap: 'overflow-x-auto',
  table: 'w-full border-collapse text-left text-sm',
  th: 'border-b border-zinc-300 px-3 py-2 font-medium',
  td: 'border-b border-zinc-200 px-3 py-2 align-top',
  pager: 'mt-4 flex items-center gap-3 text-sm',
};
```

7.7 Components

File: `components/FormBuilder.tsx`. Create-form screen.

```
'use client';

import { zodResolver } from '@hookform/resolvers/zod';
import { useRouter } from 'next/navigation';
import { useState } from 'react';
import { useFieldArray, useForm } from 'react-hook-form';
import { api, getErrorMessage } from '@/lib/fetch';
import { builder as p, page, shared } from '@/lib/responsive/pattrens/forms';
import {
  builderFormSchema,
  toCreateFormPayload,
  type BuilderForm,
} from '@/lib/schemas/builder';
import { isChoiceType, questionTypes } from '@/lib/schemas/question';
import { OptionsEditor } from './OptionsEditor';

const emptyQuestion: BuilderForm['questions'][number] = {
  label: '',
```

```
  type: 'short_text',
  required: false,
  options: [],
};

/** Create a form: react-hook-form with a field array and builderFormSchema. */
export function FormBuilder() {
  const router = useRouter();
  const [serverError, setServerError] = useState<string | null>(null);
  const {
    register,
    control,
    handleSubmit,
    watch,
    formState: { errors, isSubmitting },
  } = useForm<BuilderForm>({
    resolver: zodResolver(builderFormSchema),
    defaultValues: { title: '', description: '', questions: [emptyQuestion] },
  });
  const { fields, append, remove } = useFieldArray({
    control,
    name: 'questions',
  });

  /** Sends the payload; on success go to the dashboard. */
  async function onSubmit(values: BuilderForm) {
    setServerError(null);
    try {
      await api('@post/forms', { body: toCreateFormPayload(values) });
      router.push('/');
    } catch (error) {
      setServerError(getErrorMessage(error));
    }
  }

  return (
    <form className={p.form} onSubmit={handleSubmit(onSubmit)}>
      <div>
        <label className={shared.label}>Title</label>
        <input className={shared.input} {...register('title')} />
        {errors.title && <p className={page.error}>{errors.title.message}</p>}
      </div>
      <div>
        <label className={shared.label}>Description</label>
        <textarea className={shared.input} {...register('description')} />
      </div>

      {fields.map((field, index) => {
        const type = watch(`questions.${index}.type`);
        return (
          <div key={field.id} className={p.card}>
            <input
              className={shared.input}
              placeholder="Question"
              {...register(`questions.${index}.label`)}
            />
            {errors.questions?.[index]?.label && (
              <p className={page.error}>
                {errors.questions[index]?.label?.message}
              </p>
            )}
            <div className={p.row}>
              <select
                className={shared.input}
                {...register(`questions.${index}.type`)}
```

```
              >
                {questionTypes.map((t) => (
                  <option key={t} value={t}>
                    {t.replace('_', ' ')}
                  </option>
                ))}
              </select>
              <label className={p.row}>
                <input
                  type="checkbox"
                  className={p.checkbox}
                  {...register(`questions.${index}.required`)}
                />
                Required
              </label>
              <button
                type="button"
                className={shared.buttonGhost}
                onClick={() => remove(index)}
              >
                Delete
              </button>
            </div>
            {isChoiceType(type) && (
              <OptionsEditor
                index={index}
                control={control}
                register={register}
              />
            )}
          </div>
        );
      })}

      <button
        type="button"
        className={shared.buttonGhost}
        onClick={() => append(emptyQuestion)}
      >
        Add question
      </button>
      {serverError && <p className={page.error}>{serverError}</p>}
      <div>
        <button type="submit" className={shared.button} disabled={isSubmitting}>
          {isSubmitting ? 'Saving...' : 'Create form'}
        </button>
      </div>
    </form>
  );
}
```

File: `components/OptionsEditor.tsx`. Options editor for choice questions.

```
'use client';

import {
  useFieldArray,
  type Control,
  type UseFormRegister,
} from 'react-hook-form';
import type { BuilderForm } from '@/lib/schemas/builder';
import { builder as p, shared } from '@/lib/responsive/pattrens/forms';

type Props = {
```

```
  index: number;
  control: Control<BuilderForm>;
  register: UseFormRegister<BuilderForm>;
};

/** Choices of one question: one input per option, add, remove, Yes / No. */
export function OptionsEditor({ index, control, register }: Props) {
  const { fields, append, remove, replace } = useFieldArray({
    control,
    name: `questions.${index}.options`,
  });

  return (
    <div className="space-y-2">
      {fields.map((field, optionIndex) => (
        <div key={field.id} className={p.row}>
          <input
            className={shared.input}
            placeholder={`Option ${optionIndex + 1}`}
            {...register(`questions.${index}.options.${optionIndex}.value`)}
          />
          <button
            type="button"
            className={shared.buttonGhost}
            onClick={() => remove(optionIndex)}
          >
            Remove
          </button>
        </div>
      ))}
      <div className={p.row}>
        <button
          type="button"
          className={shared.buttonGhost}
          onClick={() => append({ value: '' })}
        >
          Add option
        </button>
        <button
          type="button"
          className={shared.buttonGhost}
          onClick={() => replace([{ value: 'Yes' }, { value: 'No' }])}
        >
          Yes / No
        </button>
      </div>
    </div>
  );
}
```

File: `components/QuestionField.tsx`. One input per question type.

```
'use client';

import { Controller, type Control, type FieldValues } from 'react-hook-form';
import { page, shared, survey as p } from '@/lib/responsive/pattrens/forms';
import type { Question } from '@/lib/schemas/question';
import { FileUploadField } from './FileUploadField';

type Props = {
  question: Question;
  control: Control<FieldValues>;
  error?: string;
};
```

```
const inputTypes: Record<string, string> = {
  short_text: 'text',
  number: 'number',
  email: 'email',
  date: 'date',
};

/** One input per question type. */
export function QuestionField({ question, control, error }: Props) {
  return (
    <div className={p.field}>
      <label className={shared.label}>
        {question.label}
        {question.required && ' *'}
      </label>
      <Controller
        control={control}
        name={question.id}
        render={({ field }) => {
          switch (question.type) {
            case 'long_text':
              return (
                <textarea
                  className={shared.input}
                  {...field}
                  value={field.value ?? ''}
                />
              );
            case 'single_choice':
              return (
                <div className={p.options}>
                  {question.options.map((option) => (
                    <label key={option} className={p.option}>
                      <input
                        type="radio"
                        checked={field.value === option}
                        onChange={() => field.onChange(option)}
                      />
                      {option}
                    </label>
                  ))}
                </div>
              );
            case 'multi_choice': {
              const selected: string[] = field.value ?? [];
              return (
                <div className={p.options}>
                  {question.options.map((option) => (
                    <label key={option} className={p.option}>
                      <input
                        type="checkbox"
                        checked={selected.includes(option)}
                        onChange={(e) =>
                          field.onChange(
                            e.target.checked
                              ? [...selected, option]
                              : selected.filter((o) => o !== option),
                          )
                        }
                      />
                      {option}
                    </label>
                  ))}
                </div>
```

```
            );
          }
          case 'file':
            return (
              <FileUploadField
                value={field.value ?? ''}
                onChange={field.onChange}
              />
            );
          default:
            return (
              <input
                className={shared.input}
                type={inputTypes[question.type] ?? 'text'}
                {...field}
                value={field.value ?? ''}
              />
            );
        }
      }}
    />
    {error && <p className={page.error}>{error}</p>}
  </div>
 );
}
```

File: `components/FileUploadField.tsx`. File picker and upload.

```
'use client';

import { useState } from 'react';
import { getErrorMessage, uploadFile } from '@/lib/fetch';
import { page, shared } from '@/lib/responsive/pattrens/forms';

type Props = { value: string; onChange: (id: string) => void };

/** Checks the file, uploads it to /api/files, stores the returned id. */
export function FileUploadField({ value, onChange }: Props) {
  const [status, setStatus] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);

  async function onPick(file: File | undefined) {
    if (!file) {
      return;
    }
    setError(null);
    setStatus('Uploading...');
    try {
      onChange(await uploadFile(file));
      setStatus(`Uploaded ${file.name}`);
    } catch (e) {
      setStatus(null);
      setError(getErrorMessage(e));
    }
  }

  return (
    <div>
      <input
        type="file"
        className={shared.input}
        onChange={(event) => onPick(event.target.files?.[0])}
      />
      {value && status && <p className={page.muted}>{status}</p>}
```

```
      {error && <p className={page.error}>{error}</p>}
    </div>
  );
}
```

File: `components/SurveyForm.tsx`. Public survey form.

```
'use client';

import { zodResolver } from '@hookform/resolvers/zod';
import { useMemo, useState } from 'react';
import { useForm, type FieldValues } from 'react-hook-form';
import { api, getErrorMessage } from '@/lib/fetch';
import { page, shared, survey as p } from '@/lib/responsive/pattrens/forms';
import type { Form } from '@/lib/schemas/form';
import { buildAnswerSchema } from '@/lib/schemas/submission';
import { QuestionField } from './QuestionField';

/** Fills in a survey with a schema built from its questions, then submits. */
export function SurveyForm({ form }: { form: Form }) {
  const [done, setDone] = useState(false);
  const [serverError, setServerError] = useState<string | null>(null);
  // The same rules the API applies, built from this form's own questions.
  const schema = useMemo(
    () => buildAnswerSchema(form.questions),
    [form.questions],
  );
  const defaults = useMemo(
    () =>
      Object.fromEntries(
        form.questions.map((q) => [q.id, q.type === 'multi_choice' ? [] : '']),
      ),
    [form.questions],
  );
  const {
    control,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<FieldValues>({
    // The generic schema is checked on the server again.
    resolver: zodResolver(schema as never),
    defaultValues: defaults,
  });

  async function onSubmit(answers: FieldValues) {
    setServerError(null);
    try {
      await api('@post/forms/:id/submissions', {
        params: { id: form.id },
        body: { answers },
      });
      setDone(true);
    } catch (error) {
      setServerError(getErrorMessage(error));
    }
  }

  if (done) {
    return <p>Thank you. Your response was recorded.</p>;
  }

  return (
    <form className={p.form} onSubmit={handleSubmit(onSubmit)}>
      {form.questions.map((question) => (
```

```
        <QuestionField
          key={question.id}
          question={question}
          control={control}
          error={errors[question.id]?.message as string | undefined}
        />
      ))}
      {serverError && <p className={page.error}>{serverError}</p>}
      <button type="submit" className={shared.button} disabled={isSubmitting}>
        Submit
      </button>
    </form>
  );
}
```

File: `components/SurveyLoader.tsx`. Loads a form for the public page.

```
'use client';

import { api } from '@/lib/fetch';
import { page } from '@/lib/responsive/pattrens/forms';
import { useLoad } from '@/lib/useLoad';
import { SurveyForm } from './SurveyForm';

/** Loads a form for the public page. */
export function SurveyLoader({ formId }: { formId: string }) {
  const { data, error, loading } = useLoad(
    () => api('/forms/:id', { params: { id: formId } }),
    [formId],
  );
  if (loading) {
    return <p className={page.muted}>Loading...</p>;
  }
  if (error || !data) {
    return <p className={page.error}>{error ?? 'Form not found'}</p>;
  }
  return (
    <>
      <h1 className={page.heading}>{data.title}</h1>
      {data.description && <p className={page.muted}>{data.description}</p>}
      <SurveyForm form={data} />
    </>
  );
}
```

File: `components/FormList.tsx`. Dashboard list.

```
'use client';

import Link from 'next/link';
import { api } from '@/lib/fetch';
import { list, page, shared } from '@/lib/responsive/pattrens/forms';
import { useLoad } from '@/lib/useLoad';

/** Dashboard list with links to the survey and its responses. */
export function FormList() {
  const { data, error, loading } = useLoad(() => api('/forms'), []);
  if (loading) {
    return <p className={page.muted}>Loading...</p>;
  }
  if (error || !data) {
    return <p className={page.error}>{error}</p>;
  }
  if (data.length === 0) {
```

```
    return <p className={page.muted}>No forms yet.</p>;
  }
  return (
    <ul>
      {data.map((form) => (
        <li key={form.id} className={list.item}>
          <div>
            <div>{form.title}</div>
            <div className={page.muted}>
              {form.questionCount} questions, {form.responseCount} responses
            </div>
          </div>
          <div className={list.actions}>
            <Link className={shared.link} href={`/f/${form.id}`}>
              Open
            </Link>
            <Link className={shared.link} href={`/forms/${form.id}/responses`}>
              Responses
            </Link>
          </div>
        </li>
      ))}
    </ul>
  );
}
```

File: `components/ResponsesView.tsx`. Responses table with paging.

```
'use client';

import { useState } from 'react';
import { api } from '@/lib/fetch';
import { page, shared, table as t } from '@/lib/responsive/pattrens/forms';
import { useLoad } from '@/lib/useLoad';

const PAGE_SIZE = 20;

/** Table with one column per question and paging; file answers are links. */
export function ResponsesView({ formId }: { formId: string }) {
  const [pageNumber, setPageNumber] = useState(1);
  const { data, error, loading } = useLoad(async () => {
    const [form, responses] = await Promise.all([
      api('/forms/:id', { params: { id: formId } }),
      api('/forms/:id/submissions', {
        params: { id: formId },
        query: { page: pageNumber, limit: PAGE_SIZE },
      }),
    ]);
    return { form, responses };
  }, [formId, pageNumber]);

  if (loading) {
    return <p className={page.muted}>Loading...</p>;
  }
  if (error || !data) {
    return <p className={page.error}>{error}</p>;
  }
  const { form, responses } = data;
  const pages = Math.max(1, Math.ceil(responses.total / PAGE_SIZE));

  return (
    <>
      <h1 className={page.heading}>{form.title}: responses</h1>
      <div className={t.wrap}>
```

```
        <table className={t.table}>
          <thead>
            <tr>
              <th className={t.th}>Submitted</th>
              {form.questions.map((q) => (
                <th key={q.id} className={t.th}>
                  {q.label}
                </th>
              ))}
            </tr>
          </thead>
          <tbody>
            {responses.items.map((item) => (
              <tr key={item.id}>
                <td className={t.td}>
                  {new Date(item.createdAt).toLocaleString()}
                </td>
                {form.questions.map((q) => {
                  const value = item.answers[q.id];
                  return (
                    <td key={q.id} className={t.td}>
                      {q.type === 'file' && typeof value === 'string' ? (
                        <a className={shared.link} href={value}>
                          Download
                        </a>
                      ) : Array.isArray(value) ? (
                        value.join(', ')
                      ) : (
                        value
                      )}
                    </td>
                  );
                })}
              </tr>
            ))}
          </tbody>
        </table>
      </div>
      <div className={t.pager}>
        <button
          className={shared.buttonGhost}
          disabled={pageNumber <= 1}
          onClick={() => setPageNumber(pageNumber - 1)}
        >
          Previous
        </button>
        <span>
          Page {pageNumber} of {pages}
        </span>
        <button
          className={shared.buttonGhost}
          disabled={pageNumber >= pages}
          onClick={() => setPageNumber(pageNumber + 1)}
        >
          Next
        </button>
      </div>
    </>
  );
}
```

7.8 Pages

File: `app/layout.tsx`. Root layout.

```
import type { Metadata } from 'next';
import './globals.css';

export const metadata: Metadata = {
  title: 'Survey Forms',
  description: 'Build a survey, share a link, collect validated responses.',
};

export default function RootLayout({ children }: LayoutProps<'/'>) {
  return (
    <html lang="en">
      {/* Browser extensions such as Grammarly add attributes before React loads. */}
      <body suppressHydrationWarning>{children}</body>
    </html>
  );
}
```

File: `app/page.tsx`. Dashboard.

```
import Link from 'next/link';
import { FormList } from '@/components/FormList';
import { list, page, shared } from '@/lib/responsive/pattrens/forms';

/** Dashboard: the "New form" button and the list of forms. */
export default function Home() {
  return (
    <main className={page.main}>
      <div className={list.header}>
        <h1 className={page.heading}>Survey forms</h1>
        <Link className={shared.button} href="/forms/new">
          New form
        </Link>
      </div>
      <FormList />
    </main>
  );
}
```

File: `app/forms/new/page.tsx`. Builder page.

```
import { Suspense } from 'react';
import { FormBuilder } from '@/components/FormBuilder';
import { page } from '@/lib/responsive/pattrens/forms';

/** The builder sits inside Suspense: its field array creates random ids. */
export default function NewFormPage() {
  return (
    <main className={page.main}>
      <h1 className={page.heading}>New form</h1>
      <Suspense fallback={<p className={page.muted}>Loading...</p>}>
        <FormBuilder />
      </Suspense>
    </main>
  );
}
```

File: `app/f/[formId]/page.tsx`. Public survey page.

```
import { Suspense } from 'react';
```

```
import { SurveyLoader } from '@/components/SurveyLoader';
import { page } from '@/lib/responsive/pattrens/forms';

/** Params are awaited inside Suspense, as Cache Components requires. */
async function Survey({ params }: PageProps<'/f/[formId]'>) {
  const { formId } = await params;
  return <SurveyLoader formId={formId} />;
}

export default function SurveyPage(props: PageProps<'/f/[formId]'>) {
  return (
    <main className={page.main}>
      <Suspense fallback={<p className={page.muted}>Loading...</p>}>
        <Survey {...props} />
      </Suspense>
    </main>
  );
}
```

File: `app/forms/[id]/responses/page.tsx`. Responses page.

```
import { Suspense } from 'react';
import { ResponsesView } from '@/components/ResponsesView';
import { page } from '@/lib/responsive/pattrens/forms';

/** Params are awaited inside Suspense, as Cache Components requires. */
async function Responses({ params }: PageProps<'/forms/[id]/responses'>) {
  const { id } = await params;
  return <ResponsesView formId={id} />;
}

export default function ResponsesPage(
  props: PageProps<'/forms/[id]/responses'>,
) {
  return (
    <main className={page.main}>
      <Suspense fallback={<p className={page.muted}>Loading...</p>}>
        <Responses {...props} />
      </Suspense>
    </main>
  );
}
```